



Ariel Soto

Escuela de Administración y Negocios
Universidad de Concepción · Campus Chillán

SEMESTRE 1 · 2026



Fundamentos de Programación para Análisis Económico

MÓDULO II · SEMANA 3 · SESIÓN 2

Vectores y Subsetting

De valores sueltos a colecciones

Sesión 1: un valor a la vez (un salario, una edad). **Ahora:** colecciones de valores — el **vector**, la estructura más básica de R.

Casi todo en R es un vector. Una columna de datos económicos —los ingresos de 1.000 hogares— es un vector.

Al terminar esta sesión podrás:

- ▶ **Crear** vectores con `c()`, `seq()` y `rep()`
- ▶ **Seleccionar** elementos por posición y por condición (subsetting)
- ▶ Operar vectores completos de una sola vez (**vectorización**)
- ▶ Aplicarlo a un cálculo económico real: salarios reales

💬 **Activación:** en Excel, para dividir 1.000 salarios por el IPC arrastras la fórmula 1.000 veces. En R es **una** línea. Hoy vemos por qué.

Bloque A — Crear vectores

`c()` — combinar valores

La función más usada de R: **combina** valores en un vector.

```
ingresos <- c(450000, 920000, 380000, 510000, 280000)
ingresos
```

```
[1] 450000 920000 380000 510000 280000
```

```
length(ingresos)      # cuántos elementos tiene
```

```
[1] 5
```

Todos los elementos de un vector comparten **un solo tipo** (Semana 3, Sesión 1). Si mezclas, R los coerciona al tipo más general.

`seq()` y `rep()` — generar secuencias

```
seq(2015, 2022)           # secuencia de años, de 1 en 1
```

```
[1] 2015 2016 2017 2018 2019 2020 2021 2022
```

```
seq(0, 1, by = 0.25)      # de 0 a 1 en pasos de 0,25 (ej: tramos)
```

```
[1] 0.00 0.25 0.50 0.75 1.00
```

```
rep("Ñuble", 3)          # repetir un valor
```

```
[1] "Ñuble" "Ñuble" "Ñuble"
```

`seq()` sirve para ejes temporales y tramos; `rep()` para construir etiquetas o grupos. El atajo `2015:2022` es equivalente a `seq(2015, 2022)`.

Bloque B — Indexación (subsetting)

Seleccionar por posición: []

El corchete [] extrae elementos según su **posición** (empezando en 1):

```
ingresos          # recordatorio del vector
```

```
[1] 450000 920000 380000 510000 280000
```

```
ingresos[1]       # el primer elemento
```

```
[1] 450000
```

```
ingresos[c(1, 3, 5)] # primero, tercero y quinto
```

```
[1] 450000 380000 280000
```

```
ingresos[-2]      # TODOS menos el segundo (índice negativo)
```

```
[1] 450000 380000 510000 280000
```

En R se cuenta desde **1**, no desde 0. Un índice negativo **excluye** esa posición.



IMAGEN: vector con casillas numeradas 1...5 y la casilla seleccionada resaltada.

Seleccionar por condición: el subsetting lógico

La forma más potente: extraer elementos que **cumplen una condición**.

```
ingresos > 400000          # vector lógico: TRUE/FALSE por elemento
```

```
[1]  TRUE  TRUE FALSE  TRUE FALSE
```

```
ingresos[ingresos > 400000]  # se queda solo con los TRUE
```

```
[1] 450000 920000 510000
```

Dentro del corchete va un **vector lógico** (Semana 3, S1). R devuelve los elementos donde la condición es `TRUE`. Esta idea es la base de `filter()` (Semana 5) y de filtrar filas de un data frame (Semana 4).

Subsetting lógico: contar y resumir

El patrón que se repetirá todo el curso:

```
sum(ingresos > 400000)      # CUÁNTOS superan 400.000
```

```
[1] 3
```

```
mean(ingresos > 400000)    # qué PROPORCIÓN los supera
```

```
[1] 0.6
```

```
mean(ingresos[ingresos > 400000])  # promedio SOLO de los que superan
```

```
[1] 626666.7
```

`sum()` y `mean()` de una condición lógica = contar y calcular proporciones. Lo usaremos para tasas de pobreza, participación, etc.

Bloque C — Operaciones vectorizadas

Operar el vector completo de una vez

R aplica las operaciones a **todos** los elementos sin escribir un bucle. Eso es la **vectorización**.

```
ingresos / 1000           # todos divididos por 1000, de una vez
```

```
[1] 450 920 380 510 280
```

```
ingresos * 1.10           # un aumento de 10% a todos
```

```
[1] 495000 1012000 418000 561000 308000
```

Esto es lo que Excel hace arrastrando la fórmula — pero aquí es una línea, sin errores de copiado y aplicable a millones de filas.

Operar dos vectores: elemento a elemento

```
salario_nominal <- c(450000, 920000, 380000)
ipc             <- c(1.000, 1.043, 1.087)      # índice de precios por período
salario_nominal / ipc                          # cada salario por su propio IPC
```

```
[1] 450000.0 882070.9 349586.0
```

Cuando dos vectores tienen el mismo largo, R opera **posición con posición**. El primer salario se divide por el primer IPC, y así.

Bloque D — Ejemplo: salarios reales

Del salario nominal al salario real

Junta toda la sesión: crear vectores, operar y seleccionar.

```
# Salarios nominales (pesos) y el IPC de cada año (base 2020 = 1)
salario <- c(450000, 470000, 500000, 540000)
ipc      <- c(1.000, 1.043, 1.087, 1.135)
años     <- 2020:2023

# Salario real: descuenta el efecto de la inflación
salario_real <- salario / ipc
round(salario_real)
```

```
[1] 450000 450623 459982 475771
```

```
# ¿En qué años el salario real superó los 460.000?
años[salario_real > 460000]
```

```
[1] 2023
```

El salario **nominal** sube todos los años, pero el **real** corrige por inflación. Vectorización + subsetting responden la pregunta económica en dos líneas.

Cierre: lo que nos llevamos hoy

Acción	Herramienta
Crear	<code>c()</code> , <code>seq()</code> , <code>rep()</code>
Seleccionar por posición	<code>v[1]</code> , <code>v[c(1,3)]</code> , <code>v[-2]</code>
Seleccionar por condición	<code>v[v > x]</code>
Contar / proporción	<code>sum(v > x)</code> , <code>mean(v > x)</code>
Operar	vectorización: <code>v / 1000</code> , <code>a / b</code>

Puente a la Semana 4: un vector es **una** columna. La próxima semana ponemos varios vectores lado a lado para formar un **data frame** — la tabla, estructura central del curso. Todo lo de hoy se aplicará a cada columna.



Sesión 2 — Fin

Al laboratorio práctico (2h)

Práctica intensiva: crear y manipular vectores de datos económicos reales **A1 (formativa, sin nota)**: script con vectores creados, manipulados y documentados. Prepara la **T2** (Semana 5)